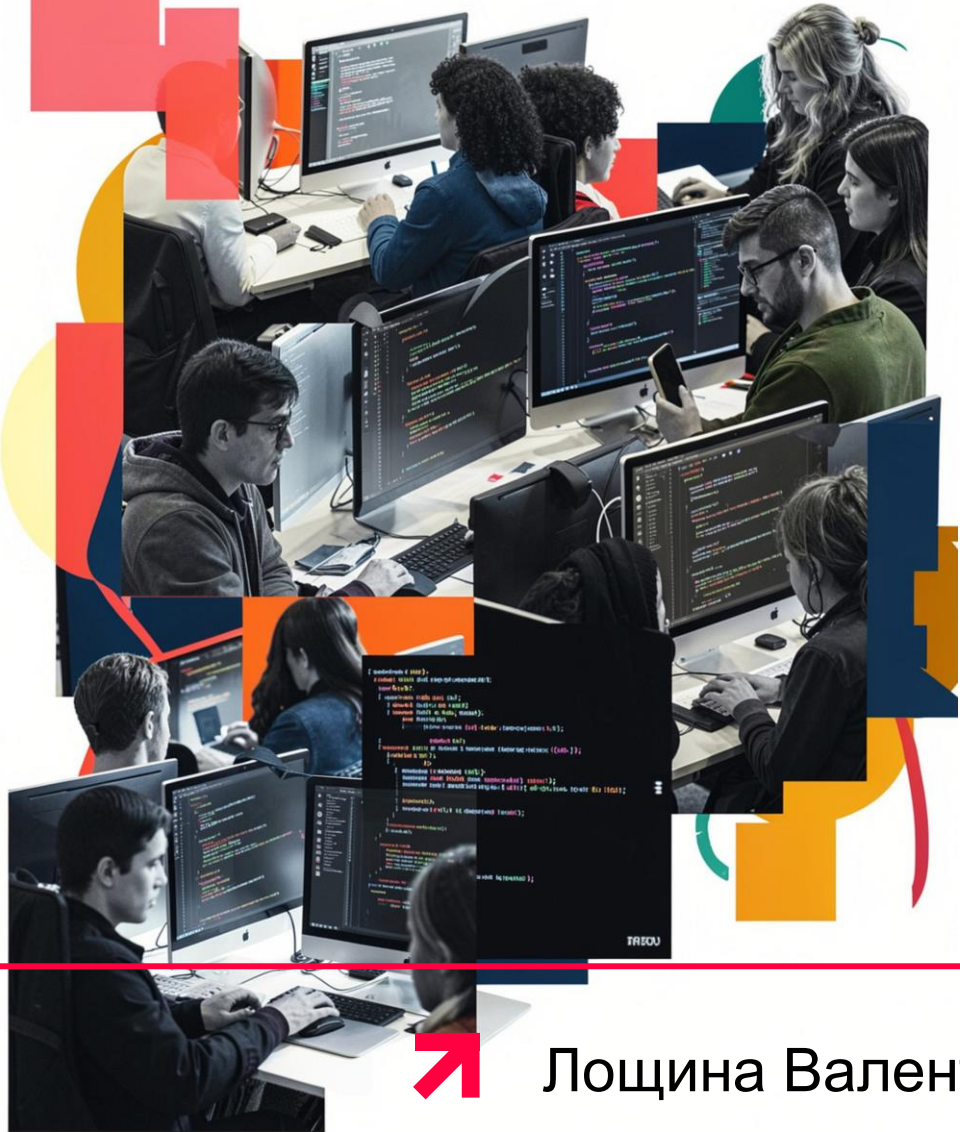




Лекция 2

«Технология разработки ПО. Жизненный цикл программного продукта»



Лощина Валентина Григорьевна

Цели и задачи лекции

Цель занятия

Сформировать у обучающихся понимание сущности технологии разработки ПО и жизненного цикла программного продукта, а также осознать их взаимосвязь в современной инженерии программного обеспечения.

Задачи

- Раскрыть понятие «технология разработки ПО»
- Изучить этапы жизненного цикла программного продукта
- Рассмотреть связь между технологией и жизненным циклом

Формируемые компетенции

- **ОК 01** — Выбирать способы решения задач профессиональной деятельности
- **ОК 02** — Использовать современные средства поиска и анализа информации

План лекции

- 1 Что такое технология разработки ПО и зачем она нужна?
- 2 Жизненный цикл программного продукта и его основные этапы.
- 3 Связь технологии и жизненного цикла
- 4 Модели жизненного цикла ПО
- 5 Вопросы для самоконтроля. Домашнее задание и литература.

 Курс построен так, что каждый модуль даёт практические навыки для портфолио. В этой индустрии **портфолио важнее диплома**.

Что такое технология разработки ПО?

Технология разработки ПО — это совокупность методов, инструментов, процессов и практик, используемых для создания программного продукта от начальной идеи до его полного вывода из эксплуатации. Простыми словами: технология отвечает на вопрос «как именно мы делаем программы?»



Методы

Подходы к анализу, проектированию, кодированию и тестированию



Инструменты

IDE, компиляторы, системы контроля версий, баг-трекеры



Процессы

Последовательность этапов, роли, артефакты



Практики

Code Review, парное программирование, CI/CD



Стандарты

ГОСТ, ISO, внутренние регламенты компании

Зачем нужна технология?

Без технологии — хаос

- Никто не знает, кто за что отвечает
- Требования теряются по ходу проекта
- Ошибки обнаруживаются слишком поздно
- Проект не укладывается в сроки и бюджет

Технология даёт

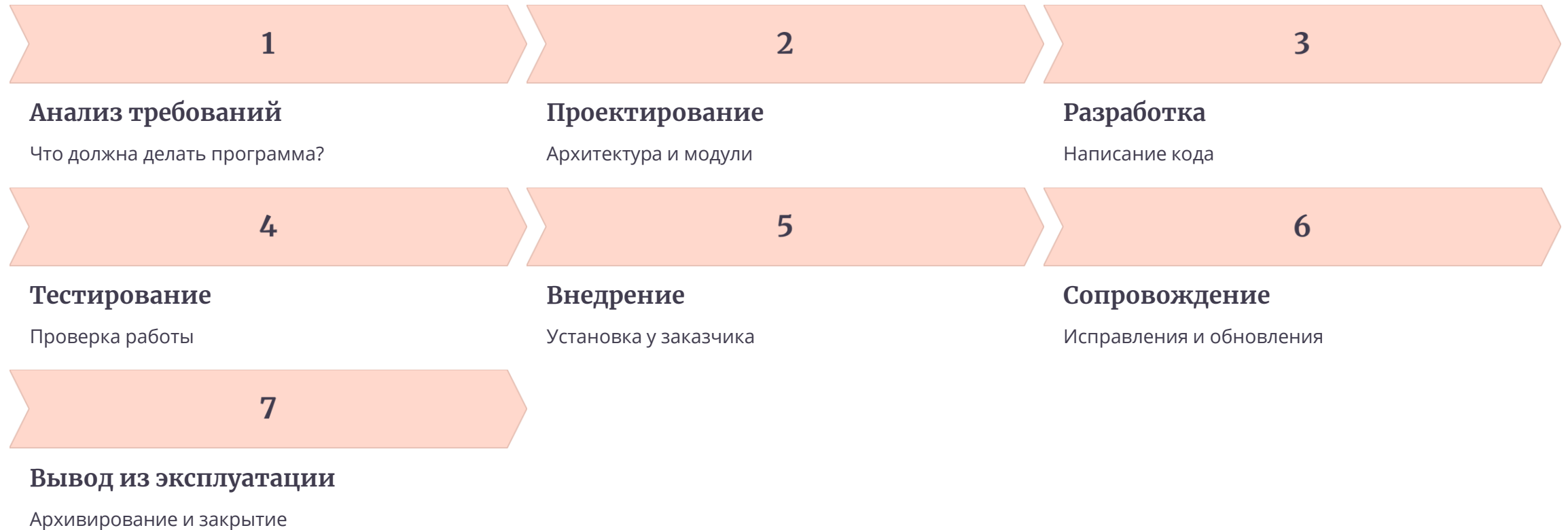
-  **Предсказуемость** — понятные сроки и результаты
-  **Управляемость** — контроль на каждом этапе
-  **Качество** — меньше ошибок, больше надёжности
-  **Командная работа** — чёткие роли и процессы
-  **Повторное использование** — накопление опыта

 Технология существует на четырёх уровнях: **личный** (как разработчик пишет код), **командный** (взаимодействие команды), **организационный** (управление проектами в компании) и **отраслевой** (стандарты ГОСТ, ISO).

Жизненный цикл программного продукта

Жизненный цикл ПО (ЖЦ ПО) — это период времени от момента появления идеи о создании программного продукта до момента его полного вывода из эксплуатации.

Важно: Жизненный цикл — это не то же самое, что «разработка». Разработка — лишь одна из частей жизненного цикла. ЖЦ охватывает всё: от первой идеи до последнего дня использования программы.



Этапы ЖЦ: от анализа до разработки

1

Анализ требований

Встречи с заказчиком, сбор и документирование требований, составление технического задания.

Артефакты: ТЗ, спецификация требований.

2

Проектирование

Разработка архитектуры, проектирование модулей, интерфейсов и базы данных.

Артефакты: архитектурная документация, UML-диаграммы.

3

Разработка (кодирование)

Написание кода, работа с системами контроля версий, Code Review.

Артефакты: исходный код, комментарии, документация.

Этапы ЖЦ: от тестирования до вывода

4

Тестирование

Модульное, интеграционное, системное тестирование. Поиск и исправление ошибок. **Артефакты:** тест-кейсы, баг-репорты, отчёты.

5

Внедрение

Установка у заказчика, обучение пользователей, миграция данных. **Артефакты:** руководство пользователя, инструкция по установке.

6

Сопровождение

Исправление ошибок, обновление, доработка, техническая поддержка. **Артефакты:** обновления, патчи, отчёты.

7

Вывод из эксплуатации

Отказ от использования, архивирование данных. **Артефакты:** акт о выводе, архив.

Связь технологии и жизненного цикла

Технология определяет, **как именно** проходят этапы жизненного цикла. Каждый этап ЖЦ опирается на конкретные методы, инструменты и стандарты, которые задаёт выбранная технология разработки.

Этап ЖЦ	Что определяет технология
Анализ требований	Методы сбора: интервью, опросы, User Stories
Проектирование	Нотации (UML, ERD), принципы (SOLID)
Разработка	Стандарты кода, инструменты (Git, IDE)
Тестирование	Виды и методы тестирования, инструменты
Внедрение	Способы развёртывания (CI/CD, Docker)
Сопровождение	Системы баг-трекинга, мониторинг

 **Ключевая мысль:** Модель ЖЦ определяет, в *каком порядке* идут этапы, а технология определяет, *как именно* каждый этап выполняется.

Модели и стандарты жизненного цикла

Основные модели ЖЦ

Каскадная

Этапы строго
последовательно, без
возврата

Инкрементная

Проект разбит на части
— инкременты

Спиральная

Каждый виток — полный
цикл + анализ рисков

Итеративная

Повторяющиеся циклы
улучшения продукта

Стандарты ЖЦ

- **ГОСТ 19.102-77** — стадии разработки программ (ТЗ → эскизный проект → технический проект → рабочая документация → внедрение → сопровождение)
- **ГОСТ 34.601-90** — стадии создания автоматизированных систем
- **ISO/IEC 12207** — процессы жизненного цикла ПО
- **ISO/IEC 15504 (SPICE)** — оценка процессов разработки

Подробнее о моделях ЖЦ — в отдельной лекции.

Каскадная модель (Waterfall)

1970 · УИНСТОН РОЙС

Этапы выполняются **строго последовательно**. Переход к следующему этапу возможен только после полного завершения предыдущего. Возврат назад не предусмотрен — это главная особенность и главное ограничение модели.

✓ Преимущества

- Простая и понятная структура
- Чёткое планирование сроков и бюджета
- Хорошо документирована
- Подходит для проектов с фиксированными требованиями

✗ Недостатки

- Нельзя вернуться на предыдущий этап
- Тестирование — в самом конце, ошибки дорого исправлять
- Заказчик видит результат только в конце
- Плохо работает при изменении требований

📌 **Когда применять:** Государственные и оборонные заказы, небольшие проекты с понятной задачей, проекты с фиксированными требованиями.



Инкрементная модель (Incremental)

1980-Е

Проект разбивается на **инкременты** — части. Каждый инкремент представляет собой полный цикл разработки (от требований до внедрения), но только для определённой части функциональности. Продукт «растёт» постепенно: сначала ядро системы, затем дополнительные функции.

Как это работает

- **Инкремент 1** — ядро системы
- **Инкремент 2** — дополнительные функции
- **Инкремент 3** — ещё функции
- И так далее до завершения

✓ Преимущества

- Заказчик видит результат после каждого инкремента
- Можно изменять требования между инкрементами
- Раннее тестирование и отладка
- Можно начать использовать продукт раньше

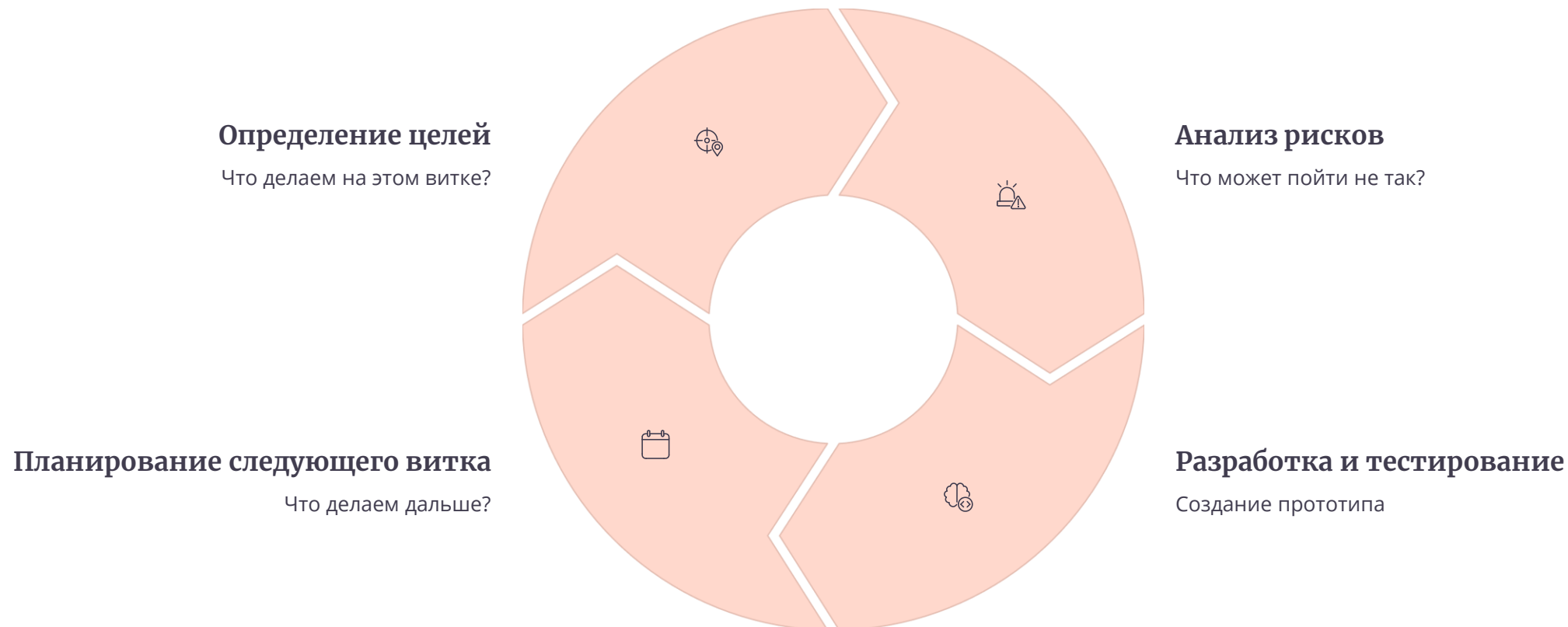
✗ Недостатки

- Требуется хорошего планирования архитектуры
- Нужно чётко определить границы каждого инкремента
- Возможны проблемы интеграции между частями

Спиральная модель (Spiral)

1986 · БАРРИ БОЭМ

Эволюция инкрементной модели, где каждый **виток спирали** — это полный цикл разработки с акцентом на **анализ рисков**. Спираль расширяется, а продукт растёт от прототипа к готовой системе.



Применение: Крупные, сложные и рискованные проекты. Системы, где важна безопасность и надёжность — медицина, авиация, финансы. Проекты с неопределёнными требованиями.

Итеративная модель (Iterative)

1990-Е

Проект разбивается на **итерации** — повторения. Каждая итерация — полный цикл разработки, но результат — это **улучшенная версия одного и того же продукта**. В отличие от инкрементной модели, где добавляются новые функции, в итеративной — совершенствуется уже существующее.



Итерация 1 — Базовая версия

Анализ → Проект → Код → Тест → Версия 1.0 (минимальная функциональность)



Итерация 2 — Улучшенная версия

Анализ → Проект → Код → Тест → Версия 1.5 (доработки по обратной связи)



Итерация 3 — Зрелый продукт

Анализ → Проект → Код → Тест → Версия 2.0 (стабильная, полноценная система)

✓ Плюсы

- Гибкость — можно менять требования
- Раннее тестирование и обратная связь
- Постепенное улучшение качества
- Снижение рисков — ошибки видны рано

✗ Минусы

- Требуется вовлеченности заказчика на каждой итерации
- Сложнее планировать общий бюджет и сроки
- Может «расползтись» по времени
- Требуется опытной команды

Сравнительная таблица моделей

Ключевые различия между четырьмя моделями жизненного цикла по основным критериям выбора.

Критерий	Каскадная	Инкрементная	Спиральная	Итеративная
Структура	Линейная	Части	Спираль	Циклы
Возврат назад	Нет	Частично	Да	Да
Обратная связь	В конце	После каждой части	На каждом витке	На каждой итерации
Анализ рисков	Слабый	Средний	Сильный	Средний
Гибкость	Низкая	Средняя	Высокая	Высокая
Стоимость изменений	Высокая	Средняя	Низкая	Низкая
Документация	Много	Средне	Много	Меньше
Подходит для	Фиксированные требования	Крупные проекты	Рискованные проекты	Меняющиеся требования

Что выбрать: шпаргалка

Выбор модели зависит от характеристик проекта, требований заказчика и уровня рисков. Используйте эту таблицу как быстрый ориентир при принятии решения.



Требования чёткие, проект небольшой

Каскадная модель — простая структура, чёткое планирование, минимум накладных расходов.

Проект большой, можно разбить на части

Инкрементная модель — заказчик видит прогресс, можно корректировать курс между инкрементами.

Проект рискованный, нужны прототипы

Спиральная модель — акцент на анализ рисков, прототипирование, подходит для медицины и авиации.

Требования меняются, нужна гибкость

Итеративная модель — быстрая обратная связь, постепенное улучшение, идеально для веб и мобильных приложений.

 **Особые случаи:** Государственный заказ с жёсткими стандартами → Каскадная. Быстрый запуск MVP → Итеративная / Agile.

Итоги, вопросы и задания

Что важно запомнить

01

Технология разработки ПО — методы, инструменты и процессы для создания программ

02

Жизненный цикл ПО — от идеи до вывода из эксплуатации; разработка — лишь часть ЖЦ

03

Этапы ЖЦ: анализ → проектирование → разработка → тестирование → внедрение → сопровождение → вывод

04

Модель ЖЦ определяет порядок этапов, **технология** — как они выполняются

Вопросы для самоконтроля

1. Что такое технология разработки ПО?
2. Из каких компонентов состоит технология?
3. Чем отличается ЖЦ от процесса разработки?
4. Перечислите основные этапы ЖЦ
5. Что такое модель жизненного цикла?
6. Как связаны технология и жизненный цикл?
7. Какие стандарты регламентируют ЖЦ ПО?
8. Зачем нужен этап сопровождения?

Домашнее задание

- Выписать определения: «технология разработки ПО», «жизненный цикл ПО», «модель ЖЦ»
- Составить таблицу: этап ЖЦ → что делаем → артефакты
- Описать ЖЦ реального программного продукта



Литература: Орлов С.А. «Технологии разработки ПО» · Sommerwill И. «Инженерия ПО» · Брауде Э. «Технология разработки ПО» · ГОСТ 19.102-77 · ISO/IEC 12207

Вопросы для самоконтроля

Проверьте усвоение материала, ответив на следующие вопросы. Они охватывают ключевые различия и особенности каждой модели.

1 Что такое жизненный цикл ПО?

Дайте определение и перечислите основные этапы.

2 Чем каскадная модель отличается от итеративной?

Сравните структуру, гибкость и подход к обратной связи.

3 В чём главное преимущество спиральной модели?

Объясните роль анализа рисков в каждом витке.

4 Почему каскадная модель плохо работает при изменении требований?

Опишите механизм возникновения проблем.

5 Что общего у инкрементной и итеративной моделей? Чем они различаются?

Сравните подход к росту продукта.

6 Для каких проектов лучше всего подходит спиральная модель?

Приведите конкретные примеры отраслей.

7 Какая модель требует наибольшего участия заказчика?

Обоснуйте свой ответ.

8 Какая модель самая дешёвая для небольших проектов с фиксированными требованиями?

Объясните, почему.



Домашнее задание и литература

Домашнее задание

1. Составить сравнительную таблицу 4 моделей по 5 критериям (самостоятельно выбранным)
2. Привести по одному реальному примеру проекта для каждой модели с обоснованием выбора
3. Ответить письменно: «Какую модель вы бы выбрали для разработки мобильного приложения доставки еды и почему?» (минимум 150 слов)

Рекомендуемая литература

- Орлов С.А. «Технологии разработки программного обеспечения»
- Соммервилл И. «Инженерия программного обеспечения»
- Брауде Э. «Технология разработки программного обеспечения»
- Боэм Б. «Spiral Model of Software Development and Enhancement» (1988)

 Следующая лекция: Гибкие методологии разработки — Agile, Scrum, Kanban.